ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería de Sistemas · Carrera de Ciencia de Datos e Inteligencia Artificial

INFORME DE PRÁCTICA N.º 1

Consolidación de reportes de ventas (CSV, JSON y Excel)

Tópico del PEA · Asignatura IDSD422 — Estadística y Programación para Ciencia de Datos I

Campo	Dato
Estudiante(s)	Tingo Pérez Alan Fernando — usuario de GitHub: AlanTingo
Docente	Ing. Christian Suarez
Período académico	2026-B
Fecha de entrega	16/10/2026
Repositorio (fork)	https://github.com/AlanTingo/IDSD422-lab1-lectura-datos

1. Introducción

RetailAndina S.A. recibe el reporte semanal de ventas en tres formatos porque provienen de tres sistemas independientes: un CSV del ERP de sucursales, un JSON de la API de la plataforma de fidelización y un Excel consolidado por Finanzas. La gerencia pidió al equipo de Business Intelligence confirmar que los tres archivos contienen la misma información, describir su estructura (filas, columnas y tipos de datos) y calcular los indicadores iniciales de la semana.

2. Objetivos

Objetivo general: analizar un mismo reporte de ventas disponible en CSV, JSON y Excel, y reportar a la gerencia si los tres son equivalentes y cuáles son los indicadores de la semana.

Objetivos específicos:

1. Leer los tres archivos en Python y en R.
2. Verificar las dimensiones y los tipos de datos de cada columna en los tres formatos.
3. Calcular la venta promedio por sucursal, el ticket promedio, la sucursal con mayor y menor venta y la región con mayor venta total.
4. Reportar las diferencias técnicas entre formatos y entre lenguajes.

3. Desarrollo de la práctica

3.1 Preparación del entorno

Hice un fork del repositorio del laboratorio en mi cuenta de GitHub (AlanTingo). Para Python trabajé en Google Colab: cloné mi fork con un token personal de GitHub, configuré mi nombre y correo en git e instalé `pandas` y `openpyxl`. Para R trabajé en Posit Cloud, creando un proyecto desde la URL de mi fork e instalando `readr`, `jsonlite` y `readxl`. En ambos entornos ejecuté los scripts desde la raíz del repositorio y subí un commit por cada script completado.

Problemas que tuve y cómo los resolví:

- Pegué por error el código R en la Terminal (que usa bash) en lugar del editor, lo que produjo errores de sintaxis. Revisé con `git status` que no hubiera dañado nada y volví a pegar el código en el editor: el editor es para escribir código y la Terminal para comandos (`Rscript`, `git`).
- Evité usar `git add .` para no subir archivos que Posit Cloud genera solo (`project.Rproj` y cambios en `.gitignore`); añadí cada script por su nombre.

3.2 Lectura del CSV (leer_csv)

Cargué `data/ventas.csv` y revisé dimensiones y primeras filas. Resultado: 5 filas y 4 columnas (`sucursal`, `region`, `ventas`, `clientes`) en ambos lenguajes.

Python

```
df = pd.read_csv("data/ventas.csv")
print("Dimensiones:", df.shape)
print(df.head())
```

R

```
df <- readr::read_csv("data/ventas.csv")
cat("Dimensiones:", dim(df), "\n")
print(head(df))
```

```
!python scripts/leer_csv.py

Dimensiones (filas, columnas): (5, 4)
             sucursal  region  ventas  clientes
0       Quito Norte  Sierra   12450       320
1         Quito Sur  Sierra    9870       265
2  Guayaquil Centro   Costa   15200       410
3   Guayaquil Norte   Costa   13980       372
4            Cuenca  Sierra    8540       210
```

Figura 1. CSV en Python: dimensiones (5, 4) y primeras filas.

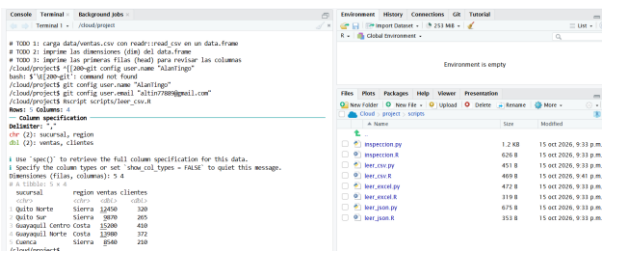


Figura 2. CSV en R: dimensiones 5 4 y tipos inferidos (chr y dbl).

3.3 Lectura del JSON (leer_json)

Primero miré el archivo crudo con `head -c 800 data/ventas.json`: empieza con [y contiene una lista de objetos, cada uno con las claves `sucursal`, `region`, `ventas` y `clientes`. En Python, `json.load` devolvió una lista (`list`) de diccionarios (`dict`), no una tabla, así que la convertí en DataFrame. En R, `jsonlite::fromJSON` devolvió directamente un `data.frame`.

Python

```
with open("data/ventas.json", encoding="utf-8")
as f:
    datos = json.load(f)   # list de dict
df = pd.DataFrame(datos)   # cada dict es una
fila
```

R

```
datos <- jsonlite::fromJSON("data/ventas.json")
# ya devuelve data.frame
df <- as.data.frame(datos)
```

```
!python scripts/leer_json.py

Tipo devuelto por json.load: <class 'list'>
Tipo de cada elemento: <class 'dict'>
Dimensiones (filas, columnas): (5, 4)
          sucursal  region  ventas  clientes
0      Quito Norte  Sierra   12450       320
1        Quito Sur  Sierra    9870       265
2  Guayaquil Centro   Costa   15200       410
3   Guayaquil Norte   Costa   13980       372
4           Cuenca  Sierra    8540       210
```

Figura 3. JSON en Python: tipos list y dict, y dimensiones (5, 4).

```
/cloud/project$ Rscript scripts/leer_json.R
Clase devuelta por fromJSON: data.frame
Dimensiones (filas, columnas): 5 4
         sucursal region ventas clientes
1      Quito Norte Sierra  12450      320
2        Quito Sur Sierra   9870      265
3 Guayaquil Centro  Costa  15200      410
4  Guayaquil Norte  Costa  13980      372
5           Cuenca Sierra   8540      210
```

Figura 4. JSON en R: clase data.frame y dimensiones 5 4.

3.4 Lectura del Excel (leer_excel)

Listé las hojas con `pd.ExcelFile(...).sheet_names` y vi que el libro tiene una sola hoja, `ventas`, con los encabezados en la primera fila y sin títulos ni totales. Aun así indiqué la hoja explícitamente al leer.

Python

```
df = pd.read_excel("data/ventas.xlsx",
                   sheet_name="ventas")
```

R

```
df <- readxl::read_excel("data/ventas.xlsx",
                         sheet = "ventas")
```

```
!python scripts/leer_excel.py

Dimensiones (filas, columnas): (5, 4)
          sucursal  region  ventas  clientes
0      Quito Norte  Sierra   12450       320
1        Quito Sur  Sierra    9870       265
2  Guayaquil Centro   Costa   15200       410
3   Guayaquil Norte   Costa   13980       372
4           Cuenca  Sierra    8540       210
```

Figura 5. Excel en Python: hoja ventas, dimensiones (5, 4).

```
/cloud/project$ Rscript scripts/leer_excel.R
Dimensiones (filas, columnas): 5 4
# A tibble: 5 × 4
  sucursal         region ventas clientes
  <chr>            <chr>   <dbl>    <dbl>
1 Quito Norte      Sierra  12450      320
2 Quito Sur        Sierra   9870      265
3 Guayaquil Centro Costa   15200      410
4 Guayaquil Norte  Costa   13980      372
5 Cuenca           Sierra   8540      210
/cloud/project$ git add scripts/leer_excel.R
```

Figura 6. Excel en R: tibble de 5 × 4.

3.5 Comparación de los tres formatos

Cargué los tres archivos en Python y comparé dimensiones y contenido con `DataFrame.equals`: `print(csv.shape, jsn.shape, xlsx.shape)` dio (5, 4) en los tres, y `csv.equals(jsn)` y `csv.equals(xlsx)` dieron `True`.

Formato	Origen	Filas	Columnas	Lectura en código
CSV	ERP de sucursales	5	4	`pd.read_csv()` / `readr::read_csv()`
JSON	API de fidelización	5	4	`json.load()` + `pd.DataFrame()` / `jsonlite::fromJSON()`
Excel	Consolidado de Finanzas	5	4	`pd.read_excel(sheet_name=)` / `readxl::read_excel(sheet=)`

```
import json, pandas as pd
csv = pd.read_csv("data/ventas.csv")
jsn = pd.DataFrame(json.load(open("data/ventas.json", encoding="utf-8")))
xlsx = pd.read_excel("data/ventas.xlsx", sheet_name="ventas")
print(csv.shape, jsn.shape, xlsx.shape)
print(csv.equals(jsn), csv.equals(xlsx))
```

```
(5, 4) (5, 4) (5, 4)
True True
```

Figura 7. Comparación en Python: los tres formatos tienen la misma forma y el mismo contenido.

3.6 Inspección de tipos e indicadores (inspeccion)

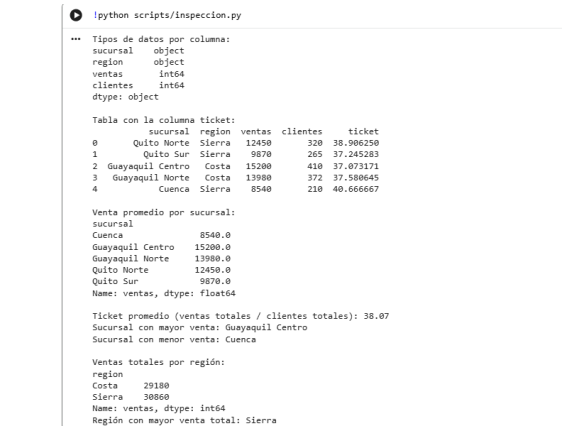
Con el CSV revisé los tipos de cada columna, creé la columna derivada `ticket = ventas / clientes` y calculé los indicadores. Para el ticket promedio usé el total de ventas dividido entre el total de clientes (no el promedio de la columna `ticket`), como pedía el script.

Python

```
df["ticket"] = df["ventas"] / df["clientes"]
df.groupby("sucursal")["ventas"].mean()
df["ventas"].sum() / df["clientes"].sum()
df.loc[df["ventas"].idxmax(), "sucursal"]
df.groupby("region")["ventas"].sum()
```

R

```
df$ticket <- df$ventas / df$clientes
aggregate(ventas ~ sucursal, df, mean)
sum(df$ventas) / sum(df$clientes)
df$sucursal[which.max(df$ventas)]
aggregate(ventas ~ region, df, sum)
```



```
!python scripts/inspeccion.py

Tipos de datos por columna:
sucursal    object
region      object
ventas       int64
clientes     int64
dtype: object

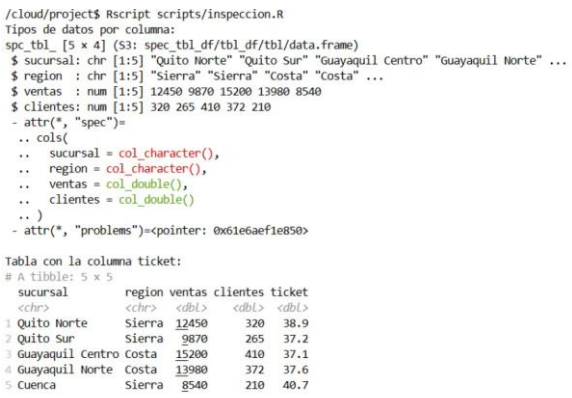
Tabla con la columna ticket:
          sucursal  region  ventas  clientes     ticket
0       Quito Norte  Sierra   12450       320  38.906250
1         Quito Sur  Sierra    9870       265  37.245283
2  Guayaquil Centro   Costa   15200       410  37.073171
3   Guayaquil Norte   Costa   13980       372  37.580645
4            Cuenca  Sierra    8540       210  40.666667

Venta promedio por sucursal:
sucursal
Cuenca              8540.0
Guayaquil Centro   15200.0
Guayaquil Norte    13980.0
Quito Norte        12450.0
Quito Sur           9870.0
Name: ventas, dtype: float64

Ticket promedio (ventas totales / clientes totales): 38.07
Sucursal con mayor venta: Guayaquil Centro
Sucursal con menor venta: Cuenca

Ventas totales por región:
region
Costa     29180
Sierra    30860
Name: ventas, dtype: int64
Región con mayor venta total: Sierra
```

Figura 8. Inspección en Python: tipos, columna ticket e indicadores.



```
/cloud/project$ Rscript scripts/inspeccion.R
Tipos de datos por columna:
spc_tbl_ [5 x 4] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ sucursal: chr [1:5] "Quito Norte" "Quito Sur" "Guayaquil Centro" "Guayaquil Norte" ...
 $ region  : chr [1:5] "Sierra" "Sierra" "Costa" "Costa" ...
 $ ventas  : num [1:5] 12450 9870 15200 13980 8540
 $ clientes: num [1:5] 320 265 410 372 210
 - attr(*, "spec")=
  .. cols(
  ..   sucursal = col_character(),
  ..   region = col_character(),
  ..   ventas = col_double(),
  ..   clientes = col_double()
  .. )
 - attr(*, "problems")=<pointer: 0x61e6aef1e850>

Tabla con la columna ticket:
# A tibble: 5 x 5
  sucursal        region ventas clientes ticket
  <chr>           <chr>   <dbl>    <dbl>  <dbl>
1 Quito Norte     Sierra  12450      320   38.9
2 Quito Sur       Sierra   9870      265   37.2
3 Guayaquil Centro Costa  15200      410   37.1
4 Guayaquil Norte Costa   13980      372   37.6
5 Cuenca          Sierra   8540      210   40.7
```

Figura 9. Inspección en R: tipos, columna ticket e indicadores (mismos valores).

3.7 Resultados

Tipos de datos inferidos

Variable	Python	R	Interpretación
`ventas`	int64	dbl (num)	Numérico: permite sumar y promediar
`clientes`	int64	dbl (num)	Numérico: conteo de clientes
`sucursal`	object	chr	Texto categórico
`region`	object	chr	Texto categórico

Los tipos fueron los esperados. Si `ventas` o `clientes` hubieran salido como texto (`object` o `chr`), habría indicado datos sucios (símbolos de moneda, separadores de miles, comas decimales, espacios o marcas como "N/D") y no se podrían sumar ni dividir sin limpiar antes.

Indicadores de la semana

Indicador	Valor
Venta promedio por sucursal	12 008 (promedio de las 5). Quito Norte 12 450; Quito Sur 9 870; Guayaquil Centro 15 200; Guayaquil Norte 13 980; Cuenca 8 540
Ticket promedio (ventas ÷ clientes)	38,07 (60 040 ÷ 1 577)
Sucursal con mayor venta	Guayaquil Centro (15 200)
Sucursal con menor venta	Cuenca (8 540)
Región con mayor venta total	Sierra (30 860), frente a Costa (29 180)

Como cada sucursal aparece una sola vez, la venta promedio por sucursal coincide con su venta de la semana. Cuenca tiene el ticket más alto (40,67) pese a ser la que menos vende; Guayaquil Centro tiene el más bajo (37,07).

3.8 Respuestas a las preguntas de la práctica

1. Filas y columnas en cada formato; ¿igual y por qué? Los tres tienen 5 filas y 4 columnas. Fue igual porque los tres sistemas exportan el mismo reporte; solo cambia la forma de almacenarlo (texto plano, lista de objetos clave-valor y libro binario con hojas). `equals` dio `True`, así que también coinciden valores y tipos.

2. Tipo inferido para ventas y clientes; ¿qué indicaría texto? Numérico: `int64` en Python y `dbl` en R. Que salieran como texto indicaría datos sucios o mal formateados que impedirían calcular totales y promedios.

3. ¿Qué devuelve `json.load` y por qué aplanarla? Devuelve estructuras nativas de Python: aquí, una `list` de `dict`. No es una tabla; un DataFrame necesita filas y columnas, por eso la convertí con `pd.DataFrame(datos)` (cada diccionario es una fila y cada clave una columna). Con JSON anidado haría falta además `pd.json_normalize`.

4. ¿Para qué sirve `sheet_name` / `sheet`? Un libro de Excel puede tener varias hojas; si no se indica, se lee la primera y podría cargarse la equivocada. Indicarla hace el código explícito y reproducible. Aquí solo había una, `ventas`.

5. Diferencia conceptual entre Python y R. En R la tabla es una estructura nativa (data.frame o tibble) y sus lectores simplifican más: `fromJSON` entregó la tabla directamente. En Python, `json.load` solo interpreta el archivo y la tabla aparece al construirla con pandas: más control, pero un paso extra. También cambió la inferencia de tipos (`int64` frente a `dbl`) y que en R cada formato requiere un paquete distinto, mientras pandas cubre CSV y Excel con la misma librería.

4. ¿Qué aprendí con esta práctica?

Ahora entiendo por qué un JSON hay que aplanarlo antes de analizarlo: `json.load` me devolvió una lista de diccionarios y no una tabla, y solo después de convertirla pude calcular algo. También aprendí a verificar que tres fuentes son equivalentes comparando dimensiones, tipos y contenido con `equals`, en lugar de suponerlo. Lo más difícil fue trabajar con dos entornos y con git por primera vez en la nube: tuve errores de ruta, un correo mal configurado y pegué código R en la Terminal. Los resolví leyendo el mensaje de error, comprobando el estado con `git status` y separando el uso del editor y el de la Terminal. La próxima vez ejecutaría y verificaría cada script antes de hacer el commit y documentaría las capturas a medida que avance, y no al final.

5. Conclusiones

1. Los tres formatos son equivalentes: cada uno tiene 5 filas y 4 columnas, y la comparación en Python dio `True` para CSV frente a JSON y CSV frente a Excel. Cumple los objetivos 1 y 2.
2. Los tipos son los esperados: `ventas` y `clientes` son numéricos y `sucursal` y `region` son texto, tanto en Python como en R. Las diferencias (`int64` frente a `dbl`, y la conversión del JSON) vienen de cada lenguaje, no de los datos. Cumple los objetivos 2 y 4.
3. En la semana analizada se vendieron 60 040 con 1 577 clientes (ticket promedio 38,07). Guayaquil Centro fue la sucursal de mayor venta (15 200), Cuenca la menor (8 540) y la región Sierra superó a Costa (30 860 frente a 29 180). Cumple el objetivo 3.

Respuesta a la gerencia: los tres reportes pueden usarse de forma intercambiable porque contienen la misma información, y se recomienda mantener esta comparación como control semanal. Sierra suma más que Costa porque tiene tres sucursales frente a dos, aunque las dos mayores ventas individuales están en Costa; conviene comparar las regiones también por venta promedio por sucursal.

Alcance: con cinco sucursales y una sola semana, estos resultados describen el período analizado y no permiten proyectar tendencias.

6. Bibliografía

1. Python Software Foundation. (2026). *json — JSON encoder and decoder*. Python Documentation. https://docs.python.org/3/library/json.html
2. Pandas Development Team. (2026). *pandas.read_csv*. Pandas Documentation. https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html
3. Pandas Development Team. (2026). *pandas.read_excel*. Pandas Documentation. https://pandas.pydata.org/docs/reference/api/pandas.read_excel.html
4. Wickham, H., Hester, J. y Bryan, J. (2026). *readr: Read rectangular text data* (Paquete de R). https://readr.tidyverse.org
5. Ooms, J. (2026). *jsonlite: A simple and robust JSON parser and generator for R* (Paquete de R). https://cran.r-project.org/package=jsonlite
6. Wickham, H. y Bryan, J. (2026). *readxl: Read Excel files* (Paquete de R). https://readxl.tidyverse.org
7. Suárez, C. (2026). *IDSD422-lab1-lectura-datos* [Repositorio de código fuente]. GitHub. https://github.com/christian-suarez1979/IDSD422-lab1-lectura-datos

Declaración de uso de IA: Usé el asistente Claude (Anthropic) como apoyo para entender los pasos del laboratorio, depurar errores de ejecución y organizar el borrador del informe. Ejecuté yo mismo todo el código en Colab y Posit Cloud, las capturas son de mi propia ejecución y revisé y puedo explicar cada parte de la entrega.